SIM Global Education

Topic 4 - CM2040 Building web apps with Node and Express Part 3

Coursera Week 4

- Loh Kwong Khuin
- Associate Lecturer

© 2024 Singapore Institute of Management Pte Ltd

Learning Outcomes

- Describe and apply routing in web server development.

- Describe and apply the 'separation of concerns' programming principle in your web application development.

© 2024 Singapore Institute of Management Pte Ltd

Separation of Concerns

- **Separation of Concerns (SoC)** is a fundamental principle in software engineering and design aimed at breaking down complex systems into smaller, more manageable parts.
- The goal is to organize a system's components in a way that each part addresses a single concern, or a cohesive aspect of functionality, rather than mixing multiple concerns together.
- This approach enhances modularity, maintainability, and scalability of software systems.

© 2024 Singapore Institute of Management Pte Ltd

Separation of Concerns

Separation: The word “separation” emphasizes the act of isolating or distinguishing different concerns within a system. By separating concerns, software engineers aim to create clearer boundaries and reduce the interdependence between different parts of the system.

Concerns: In the context of software, a “concern” represents a specific aspect of functionality, behaviour, or responsibility within the system. For example, concerns might include user interface presentation, data storage and retrieval, business logic processing, error handling, security, etc. Each concern represents a cohesive aspect that can be managed independently.

© 2024 Singapore Institute of Management Pte Ltd

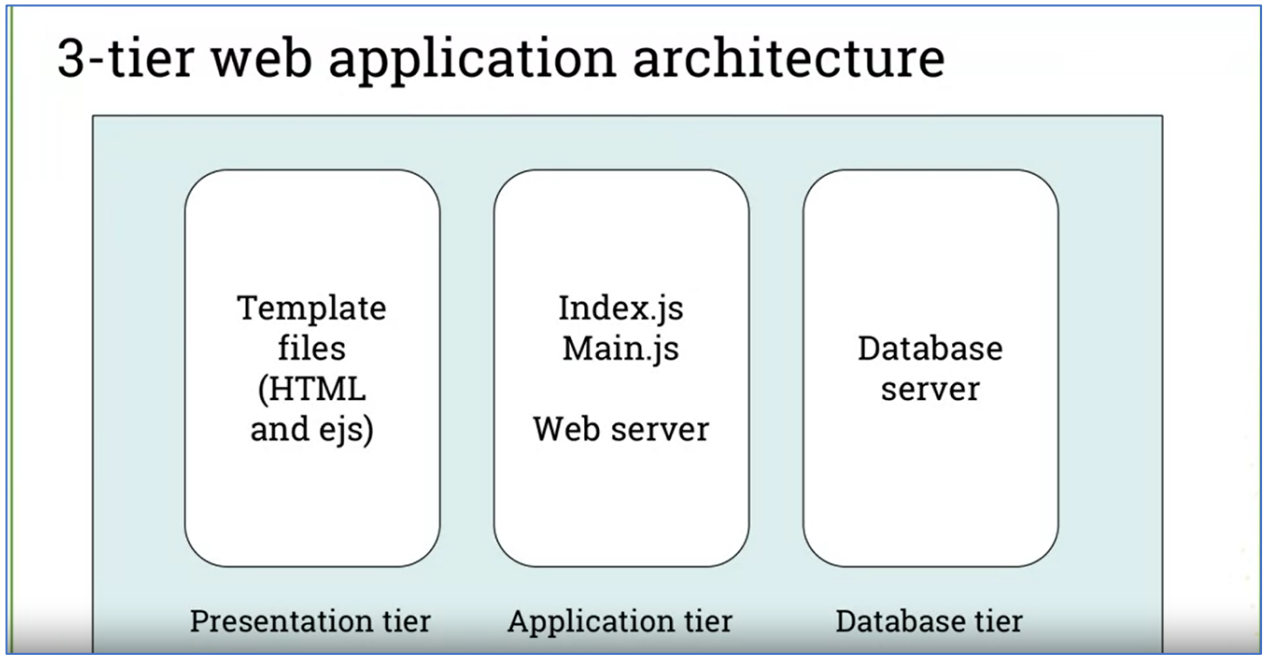
Benefits of Separation of Concern

SIM Global Education

- Separation of concerns is a Node.js architecture that ensures code readability, easy refactoring, and good code collaboration.
- Using the separation of concerns principle, you can ensure that the final system is stable and maintainable.
- The principle of separation of concerns ensures that components are not duplicated, making the system easier to maintain and refactor.
- Separation of Concerns principle aids in code reusability. This reduces maintenance costs and time by making it simple to determine where a fault is coming from and how to isolate it from the entire system.

© 2024 Singapore Institute of Management Pte Ltd

Separation of Concern Principle in Architecture



In the 3-tier web application, we have presentation tier, application tier and database tier.

We need to apply the SOC principle in structuring our web application. Gradually, we will learn to segregate

- Handle front ends as views
- Handle the front ends as routes in our web server for client-server communication
- Handle the data tier as connection to physical database

© 2024 Singapore Institute of Management Pte Ltd

Routing - Separation of Concerns Principle

SIM Global Education

- In the past, we crammed everything into one index.js file. However, as complexity grows, it becomes not feasible.
- We apply the SOC principle in re-structuring our web application by introducing:
  - Route handlers → Routes folder
  - HTML files → Views folder
- By applying SOC, it improves maintainability and scalability in our web application as addition files and routes can be added at any time later.

© 2024 Singapore Institute of Management Pte Ltd

Html files- Separation of Concerns Principle

How do we apply SOC principles in our development of the 3-tier web application?

- HTML file, presentation
- Main.js, logic including routing
- Index.js web server

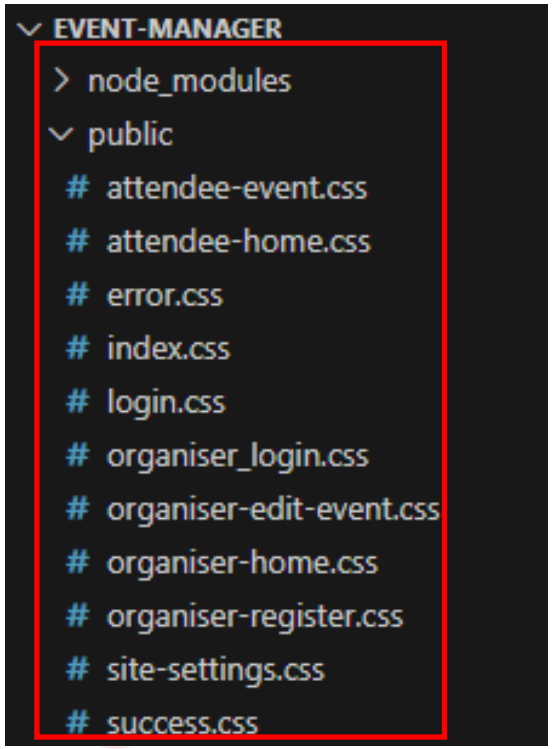
HTML files are responsible, for the presentation folder section of the code are stored in the views

The web server file, index.js is in the home directory and the main.js which is responsible for the main logic of the application , includes routing and is stored in the root directory

© 2024 Singapore Institute of Management Pte Ltd

Separation of Concerns Principle

Since we are handling the 3 tiers within our NodeJS application, we apply the SOC principles to organise our structure



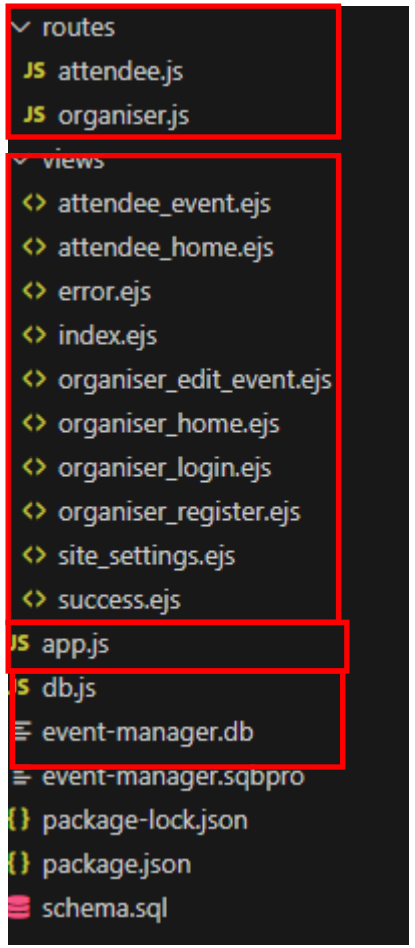
The public folder is normally reserved for static files like css files

The route folder is reserved for routes to serve the HTTP request and response

The views folder is reserved for the front-end templates for clients to request from the server

The application ties is normally represented by a standalone app.js/index,js

The database tier is also represented on its own



© 2024 Singapore Institute of Management Pte Ltd

Routing in Node.js

- **Routing defines how the client requests are handled by the application endpoints. Routing in Node refers to the process of determining how an application responds to client requests to different endpoints (URLs).**

- In a web application, these endpoints typically correspond to different pages or functionalities within the application.

- Node.js, along with frameworks like Express, provides a way to define routes, which are responsible for handling specific HTTP requests and sending back appropriate responses.

© 2024 Singapore Institute of Management Pte Ltd

Why do we use routes in NodeJS?

SIM Global Education

Routes help organize the server logic. Instead of putting everything in one place, we split behavior by what the app should do when a user hits certain URLs.

```
const express = require('express');
const app = express();

// GET route
app.get('/', (req, res) => {
  res.send('Hello World!');
});

// POST route
app.post('/submit', (req, res) => {
  res.send('Data received!');
});

// Route with a parameter
app.get('/user/:id', (req, res) => {
  res.send(`User ID is ${req.params.id}`);
});

// Start server
app.listen(3000, () => {
  console.log('Server running on http://localhost:3000');
});
```

app.get('/'): Route for HTTP GET requests to /

app.post('/submit'): Route for HTTP POST requests to /submit

req is the request object, which holds info like URL params, query strings, body, headers, etc.

res is the response object, used to send data back to the client

© 2024 Singapore Institute of Management Pte Ltd

Routes management

In Node.js, managing routes directly in a main.js file is a common approach for small applications.

However, as the application grows in complexity, developers often switch to modular routing to improve the organization and maintainability of the codebase.

Modular routing in Node.js refers to the practice of organizing and managing routes in a modular way, often by separating route definitions into different files or modules. This helps keep your codebase clean, maintainable, and scalable, especially as your application grows.

© 2024 Singapore Institute of Management Pte Ltd

All Routes in main.js (Non-modular approach)

SIM Global Education

Pros:

- Simplicity: Easy to set up for small projects with fewer routes.
- Quick to implement: No need for additional files or complex structure.

Cons:

- Scalability issues: As the app grows, the main.js file becomes hard to manage.
- Harder to maintain: All logic is in one file, making it difficult to isolate features or debug.
- Reduced readability: Large file with mixed concerns (routes, middleware, etc.) becomes less readable.

© 2024 Singapore Institute of Management Pte Ltd

Modular Routing

SIM Global Education

Pros:

- Better organization: Routes are grouped by features (e.g., auth.js, user.js), improving clarity and structure.
- Scalability: Easily extendable as the app grows by adding new route modules.
- Easier to maintain: Each module is focused on a specific concern, which improves code readability and organization.
- Improved testing: Each module can be tested independently.

Cons:

- Complexity: Additional setup required (more files and imports).
- Overhead: For small projects, splitting routes may introduce unnecessary complexity.

© 2024 Singapore Institute of Management Pte Ltd

Express Router() function

SIM Global Education

ExpressJS comes with a *Router()* function for helping in organizing the routes of your express app. This function is actually used as a middleware for distributing the routes across multiple files.

It is basically used for writing modular routes or endpoints for your Express application.

Instead of putting all the routes and their sub-routes in a single file, but it will become a nightmare to maintain this code as the complexity increases.

© 2024 Singapore Institute of Management Pte Ltd

Express Router() function

SIM Global Education

express.Router() is a super useful feature in Express for organizing your routes. Instead of defining all your routes in a single file, Router() lets you modularize them into separate files or sections

```
// routes/userRoutes.js
const express = require('express');
const router = express.Router();

// Define routes
router.get('/', (req, res) => {
  res.send('All users');
});

router.get('/:id', (req, res) => {
  res.send(`User ID: ${req.params.id}`);
});

module.exports = router;
```

express.Router() creates a mini Express app just for routes.You define routes on the router object.

```
// app.js or server.js
const express = require('express');
const app = express();

const userRoutes = require('./routes/userRoutes');

// Use the router
app.use('/users', userRoutes);

app.listen(3000, () => {
  console.log('Server running on http://localhost:3000');
});
```

app.use('/users', userRoutes) mounts all the userRoutes so:
GET /users → gets all users
GET /users/:id → gets a specific user

© 2024 Singapore Institute of Management Pte Ltd

Postman

Postman is a popular software tool used by developers for building, testing, and managing APIs (Application Programming Interfaces).

It provides a user-friendly interface to send HTTP requests (like GET, POST, PUT, DELETE) to servers, receive responses and inspect them.

https://www.postman.com/downloads/



© 2024 Singapore Institute of Management Pte Ltd

Lab/Activity/Case Study

a) Lab_05 Adding routes to your web server
b) Activity 4a
c) Activity 4b
d) Activity 4c

© 2024 Singapore Institute of Management Pte Ltd

SIM Global Education

Learn for Life
Thrive for Life

Thank you